Relatório Técnico — MLP para execução de emendas parlamentares

Identificação

Item	Descrição
Disciplina	Deep Learning — Especialização em IA Aplicada, IFG
Frente escolhida	3.1 — Multilayer Perceptrons
Integrantes	Juliano Batista · Leandro Nascimento
Dados — emendas	Portal da Transparência: Emendas Parlamentares (CGU)
Dados — municípios	IBGE Cidades@ (indicadores municipais)
Código	[link do repositório GitHub] — notebook `TP_DeepLearning_MLP_Emendas.ipynb`

1. Introdução e objetivo

O objetivo é prever, no momento em que a emenda é destinada a um município, se ela terá **execução financeira**. Uma emenda parlamentar só chega ao cidadão se o recurso for empenhado e pago, e uma parcela relevante delas nunca sai do papel.

A previsão é útil para gestores municipais, órgãos de controle e para a própria sociedade: um escore de risco de não execução permite priorizar o acompanhamento das emendas mais propensas a ficar paradas.

- **Tipo de problema:** classificação binária.
- **Unidade de observação:** um registro de emenda destinada a um município, com sua classificação orçamentária (tipo, função, subfunção, programa, ação e plano orçamentário).
- **Variável alvo `executada`:** 1 quando `Valor Pago > 0` **ou** `Valor Restos a Pagar Inscritos > 0`; 0 quando ambos são zero.
- **Entradas:** 10 atributos categóricos da emenda e da localidade e 25 indicadores socioeconômicos e fiscais do município de destino.

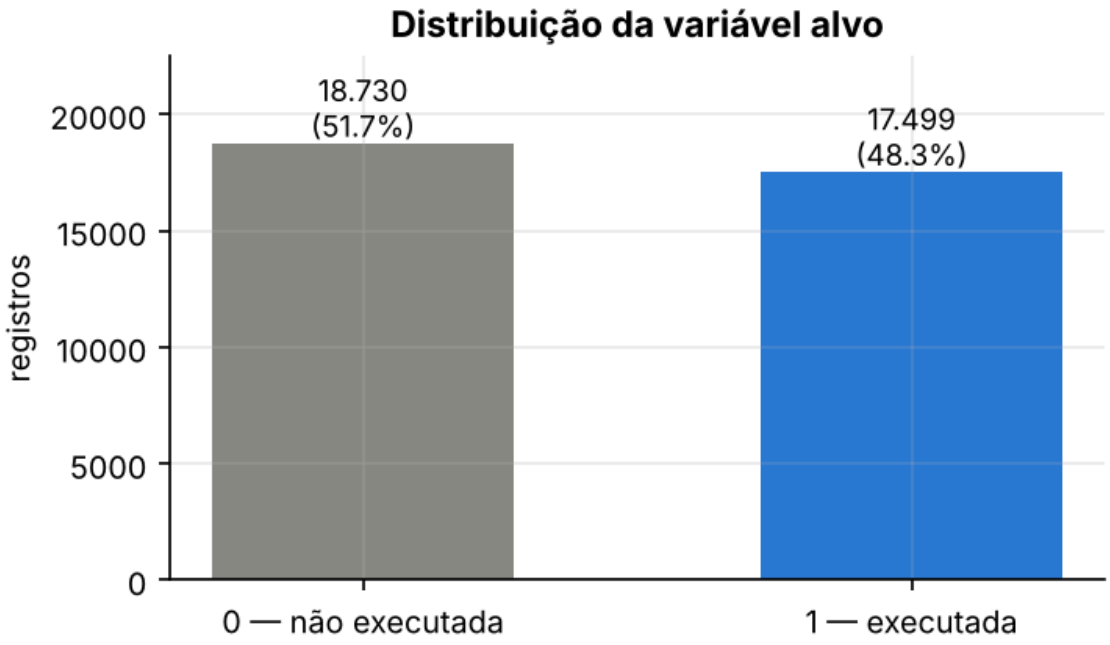
A inscrição em restos a pagar entra no alvo porque indica que a despesa foi empenhada e reconhecida como obrigação. Usar só o valor pago classificaria como não executadas emendas que estão na fase final de execução.

2. Fonte e caracterização dos dados

O conjunto final tem **36.229 registros de emendas destinadas a 5.416 municípios** (97% dos municípios do país), todas propostas antes de 2025. Os registros de 2025 e 2026 foram removidos no tratamento inicial porque ainda não tiveram tempo de completar o ciclo de execução.

Os dados de emendas vêm do Portal da Transparência (CGU) e foram cruzados pelo código IBGE do município com 25 indicadores do IBGE Cidades@ (população, área, PIB per capita, receitas e despesas municipais, educação, saúde, saneamento e urbanização).

2.1 Variável alvo



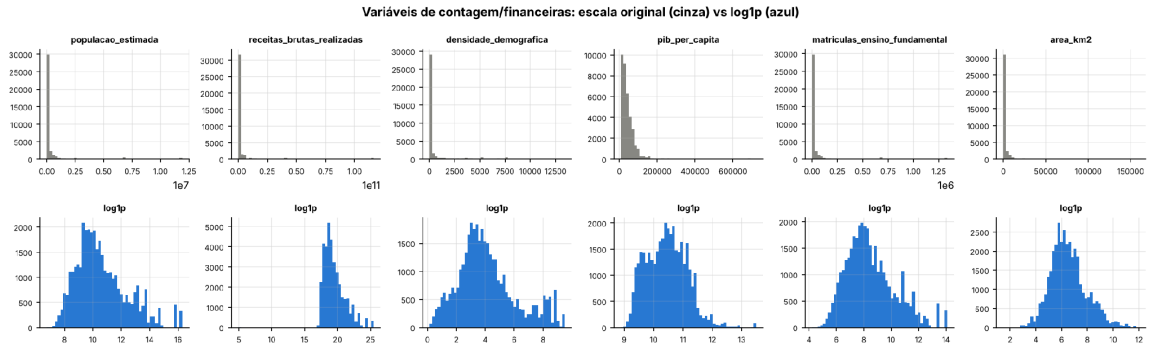
As classes estão quase balanceadas: 17.499 executadas (48,3%) e 18.730 não executadas (51,7%). Dos registros, 32,7% têm valor pago, 19,0% têm restos a pagar inscritos e 3,3% têm os dois. Dois registros têm restos a pagar negativos (estornos) e caem na classe 0.

2.2 Qualidade dos dados

Verificação	Resultado	Tratamento
Valores ausentes	10 colunas municipais, no máximo 68 registros (0,19%)	mediana (numéricas) e moda (categóricas), ajustadas no treino
Valores extremos (regra 1,5×IQR)	15–17% dos registros em população, receitas, densidade e contagens	mantidos: são municípios grandes, valores legítimos; o log1p reduz a influência

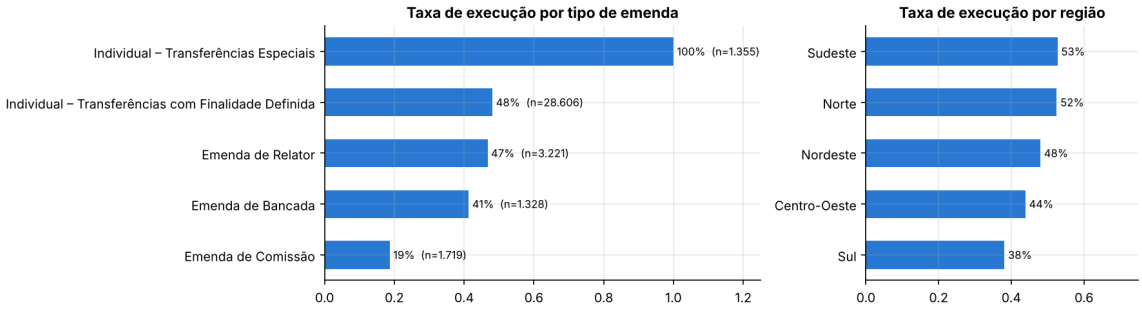
Verificação	Resultado	Tratamento
Percentuais acima de 100%	4 registros em transferências correntes, 18 em população ocupada	mantidos e registrados como limitação da fonte
Redundância	correlação de 0,9998 entre população do censo e estimada	mantida; a regularização da rede absorve a colinearidade
Alta cardinalidade	313 códigos de ação, 149 de programa	one-hot com agrupamento de categorias com menos de 10 ocorrências no treino

2.3 Distribuições

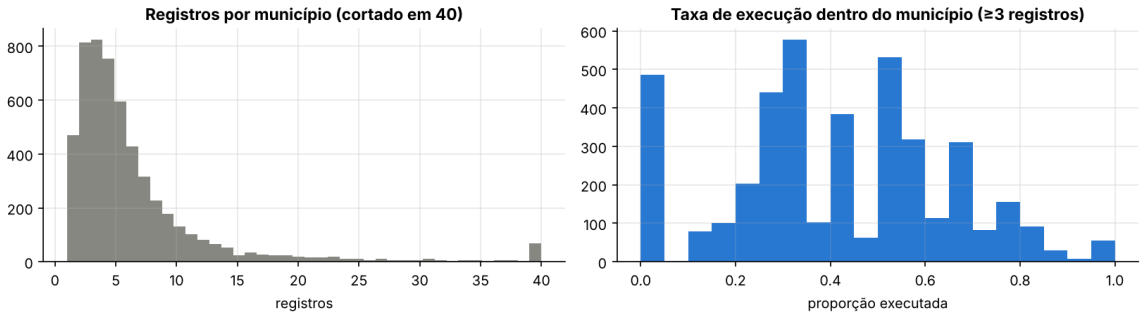


População, receitas, despesas, área, densidade e contagens de escolas e matrículas têm cauda longa dominada por capitais. Após `log1p`, a assimetria de 18 dessas variáveis cai abaixo de 1,3 em módulo (por exemplo, receitas brutas passam a 0,57). Indicadores percentuais, como escolarização e esgotamento, são apenas padronizados.

2.4 Alvo por categoria e por município



As **transferências especiais** são executadas em 100% dos 1.355 casos, porque o repasse é direto e dispensa convênio. Esse é um sinal legítimo, conhecido no momento da emenda. As emendas de **comissão** têm a menor taxa (18,7%). Entre regiões, a taxa varia de 38,1% (Sul) a 52,7% (Sudeste).



Cada município aparece em média em 6,7 registros (mediana 4; até 456 no Rio de Janeiro). Os 25 indicadores municipais se repetem em todos esses registros. Por isso a divisão dos dados foi feita por município (seção 3.3).

3. Metodologia

3.1 Seleção de atributos e vazamento

Coluna	Decisão	Motivo
Valor pago, restos a pagar inscritos	removidas das entradas	definem o alvo
Valor empenhado, liquidado, restos a pagar pagos e cancelados	removidas no tratamento inicial	são etapas da própria execução: vazamento direto
Ano, autor, número e nomes (função, ação, programa)	removidas no tratamento inicial	identificadores ou textos redundantes com os códigos
Código IBGE do município	fora das entradas	usado apenas para agrupar a divisão
Mortalidade infantil, colunas de ano de referência	removidas no tratamento inicial	inconsistências na fonte
Tipo, região, bioma, sistema costeiro, UF e 5 códigos orçamentários	categóricas	códigos numéricos não têm relação de ordem
25 indicadores municipais	numéricas	perfil socioeconômico e fiscal do destino

Todos os atributos mantidos são conhecidos quando a emenda é aprovada no orçamento, antes de qualquer execução.

3.2 Métricas

A métrica principal é a **Macro-F1**, que pesa igualmente as duas classes e penaliza um modelo que favoreça a classe majoritária. Como complementares, reportamos acurácia, precisão, revocação e F1 da classe executada, AUC-ROC, log-loss e matriz de confusão. O custo é medido por tempo de treino, tempo de inferência por amostra e número de parâmetros treináveis.

3.3 Divisão treino / validação / teste

Como os indicadores municipais se repetem em todos os registros de um município, uma divisão aleatória deixaria o mesmo município no treino e no teste, e o modelo poderia memorizá-lo. Usamos `StratifiedGroupKFold` com o município como grupo: 1/7 dos municípios vai para teste e, do restante, 1/6 para validação.

Partição	Registros	% dos registros	Municípios	% executada
Treino	25.867	71,4%	3.875	48,3%
Validação	5.175	14,3%	749	48,3%
Teste	5.187	14,3%	792	48,3%

Nenhum município aparece em mais de uma partição, o que é verificado por asserção no código. O teste é usado uma única vez, depois de toda a seleção.

3.4 Pré-processamento

Um `ColumnTransformer` do scikit-learn é ajustado **somente no treino** e aplicado à validação e ao teste:

1. 18 numéricas assimétricas e não negativas: imputação pela mediana, `log1p` e `StandardScaler`.
2. 7 numéricas percentuais ou simétricas: imputação pela mediana e `StandardScaler`.
3. 10 categóricas: imputação pela moda e `OneHotEncoder` com `min_frequency=10` e `handle_unknown="infrequent_if_exist"`, para que categorias raras e códigos novos na validação ou no teste caiam numa coluna de "infrequentes".

O resultado é um vetor de **313 entradas** (25 numéricas e 288 colunas one-hot).

Experimento 0 — variantes de pré-processamento. Com uma MLP fixa (2×128, ReLU, Adam 1e-3, lote 256, parada antecipada) e 3 sementes, comparamos três variantes na validação:

Variante	Macro-F1 (média ± dp)	AUC
só StandardScaler	0,8834 ± 0,0061	0,9580
log1p + StandardScaler	0,8803 ± 0,0031	0,9574
log1p + RobustScaler	0,8827 ± 0,0021	0,9582

As diferenças ficam dentro de um desvio-padrão entre sementes. Mantivemos log1p + StandardScaler, que é a escolha teoricamente mais adequada para as caudas longas. A pouca sensibilidade já sugere o que a seção 6.3 confirma: o desempenho depende pouco das variáveis numéricas.

3.5 Modelos de referência

Modelo	Macro-F1 treino	Macro-F1 validação	AUC validação	Treino (s)
Classe majoritária	0,3408	0,3408	0,5000	0,001
Regressão logística	0,8884	0,8701	0,9457	3,5
Gradient boosting (HistGradientBoosting)	0,9126	0,8761	0,9571	3,8

A regressão logística equivale a uma rede sem camadas ocultas e é o piso que a MLP precisa superar. O gradient boosting é a referência mais forte conhecida para dados tabulares.

4. Arquitetura e hiperparametrização

4.1 Arquitetura da MLP

A rede é construída em Keras (TensorFlow 2.21) a partir de uma função parametrizada:

- **Entrada:** 313 atributos pré-processados.
- **Camadas ocultas:** de 1 a 5 camadas `Dense`. No formato *constante*, todas têm o mesmo número de neurônios; no formato *funil*, cada camada tem metade da anterior (mínimo 8).
- **Em cada camada oculta:** penalidade L1/L2 nos pesos (opcional), Batch Normalization antes da ativação (opcional), ativação ReLU ou ELU, e dropout (opcional).
- **Saída:** 1 neurônio com sigmoide, que estima P(executada).
- **Perda e otimizador:** entropia cruzada binária com Adam.
- **Parada antecipada:** monitora a perda de validação e restaura os pesos da melhor época.

Um callback calcula a Macro-F1 de treino e de validação a cada época, o que alimenta as curvas e a poda do Optuna.

4.2 Espaço de busca

Hiperparâmetro	Intervalo	Justificativa
Número de camadas	1 a 5	cobre de rede rasa a profunda; acima de 5 há pouco ganho esperado em dados tabulares
Neurônios na 1ª camada	32, 64, 128, 256, 512	escala log2, de rede estreita a larga para 313 entradas
Formato	constante ou funil	como distribuir a capacidade entre as camadas

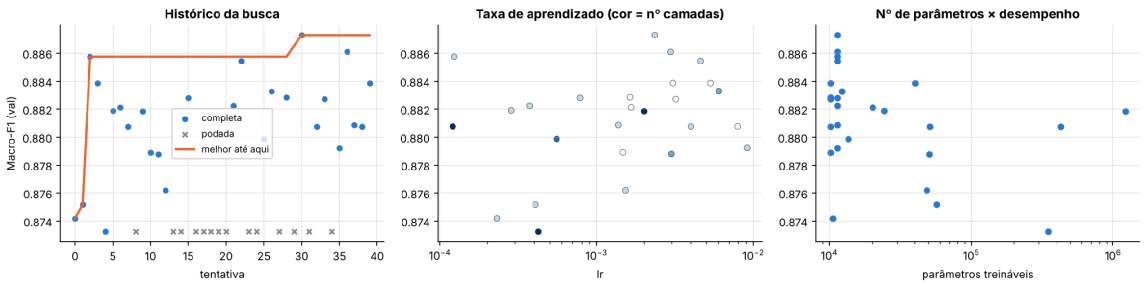
Hiperparâmetro	Intervalo	Justificativa
Ativação	ReLU ou ELU	ELU evita neurônios que param de aprender
Dropout	0 a 0,5	acima de 0,5 a rede tende ao subajuste
L2	1e-6 a 1e-2 (escala log)	de penalidade quase nula a forte
Batch Normalization	sim ou não	estabiliza o treino de redes mais profundas
Taxa de aprendizado	1e-4 a 1e-2 (escala log)	o hiperparâmetro de maior efeito esperado no Adam
Tamanho do lote	64, 128, 256, 512	ruído do gradiente versus custo por época
Paciência da parada	5, 10, 20 épocas	critério de parada antecipada

Ficaram fixos, por terem baixo efeito esperado ou serem exigidos pelo problema: otimizador Adam, saída sigmoide, perda BCE e teto de 150 épocas.

4.3 Protocolo da busca

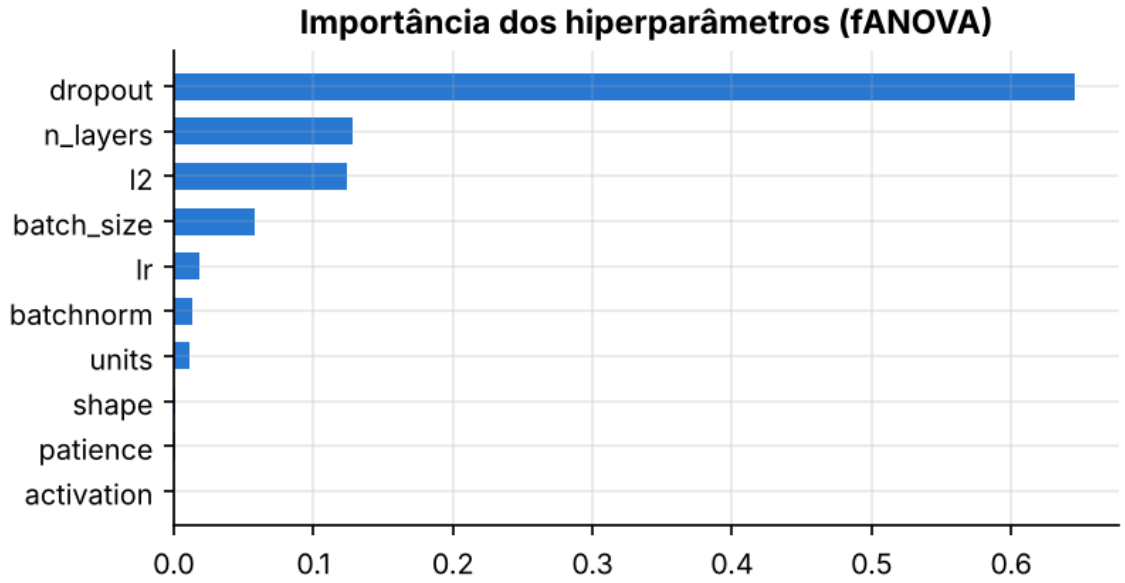
- **Ferramenta:** Optuna 5.0, amostrador TPE (bayesiano) com semente fixa.
- **Objetivo:** maximizar a Macro-F1 na **validação**, com limiar 0,5. O teste não participa.
- **Tentativas:** 40.
- **Poda:** `MedianPruner`, que interrompe uma tentativa quando sua Macro-F1 de validação fica abaixo da mediana das anteriores na mesma época (após 8 tentativas completas e 10 épocas de aquecimento).

4.4 Resultado da busca



Das 40 tentativas, 26 completaram e 14 foram podadas. As podadas pararam em média com 12 épocas, contra 47 das completas, e somaram só 3,7 dos 24,6 minutos de treino. A melhor tentativa (nº 30) atingiu **Macro-F1 de 0,8873 e AUC de 0,9610** na validação com uma rede pequena:

Hiperparâmetro	Melhor valor
Camadas × neurônios	2 × 32, constante
Ativação	ReLU
Dropout	0,26
L2	2,4e-5 (fraco)
Batch Normalization	sim
Taxa de aprendizado	0,0024
Lote	512
Paciência	20 épocas
Parâmetros treináveis	11.329

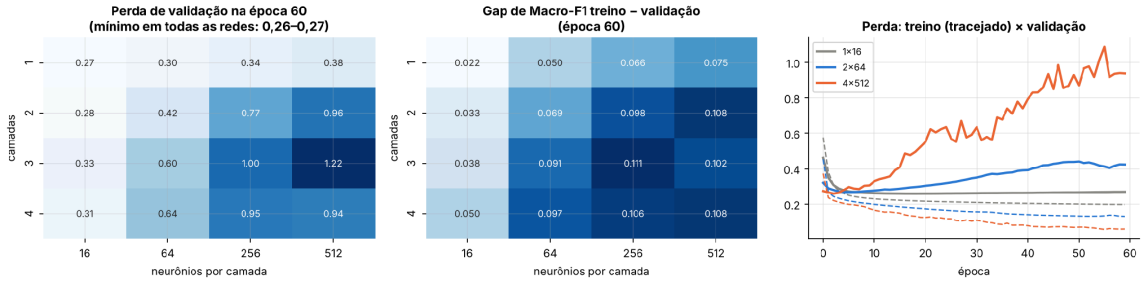


O **dropout** explica 65% da variação da Macro-F1 entre tentativas, seguido de número de camadas (13%) e L2 (12%). Tentativas com dropout abaixo de 0,07 ficaram entre 0,873 e 0,882; as quatro melhores têm dropout entre 0,25 e 0,30. As cinco melhores usam 32 neurônios e 1 ou 2 camadas. Batch Normalization aparece em 17 das 26 completas, com média um pouco maior (0,882 contra 0,880). Taxa de aprendizado, largura, formato e ativação quase não mudam o resultado dentro dos intervalos testados: o problema pede mais regularização do que capacidade.

5. Resultados de validação

5.1 Largura × profundidade (capacidade)

Para isolar o efeito da capacidade, treinamos 16 redes sem regularização e sem parada antecipada por 60 épocas: 1 a 4 camadas × 16, 64, 256 ou 512 neurônios, com a taxa de aprendizado (0,0024) e o lote (512) da melhor tentativa do Optuna.



- **A melhor Macro-F1 de validação quase não muda com a capacidade:** fica entre 0,881 e 0,889 em todas as 16 redes, e a perda mínima de validação entre 0,26 e 0,27. Uma rede de 1×16 (5.041 parâmetros) já captura o sinal.
- **O overfitting cresce com a capacidade.** Ao final das 60 épocas, a perda de validação vai de 0,27 (1×16) a 1,22 (3×512), e o gap de Macro-F1 entre treino e validação vai de 0,022 a 0,111.
- **As curvas mostram o padrão clássico:** na rede 4×512, a perda de treino cai até 0,06 enquanto a de validação atinge o mínimo por volta da época 3 e depois sobe sem parar. A rede 1×16 mantém as duas curvas próximas.
- **O custo cresce com a capacidade:** de 5 mil a 950 mil parâmetros e de 24 s a 126 s de treino, sem ganho de desempenho.

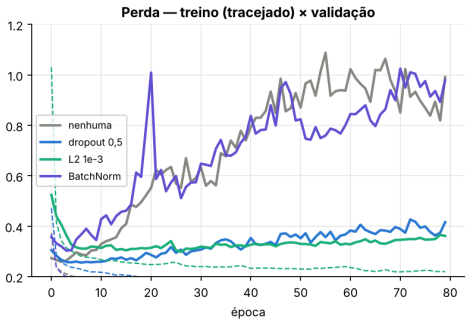
Nenhuma rede mostrou subajuste: mesmo a menor chegou a Macro-F1 de treino de 0,907. O problema não pede capacidade; pede controle do overfitting.

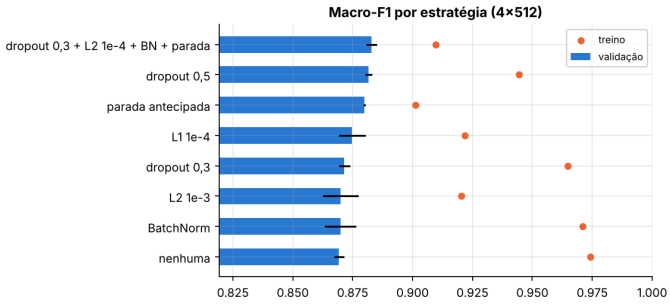
5.2 Estratégias de regularização

Partimos da rede 4×512 (cerca de 950 mil parâmetros), que teve overfitting forte, e comparamos estratégias isoladas e combinadas, com 80 épocas e 3 sementes cada.

Estratégia	Macro-F1 val. (média ± dp)	AUC val.	Log-loss val.	Macro-F1 treino	Gap	Épocas	Treino (s)
Dropout 0,3 + L2 1e-4 + BatchNorm + parada	**0,8829 ± 0,0022**	**0,9599**	**0,260**	0,910	0,027	24,7	77

Estratégia	Macro-F1 val. (média ± dp)	AUC val.	Log-loss val.	Macro-F1 treino	Gap	Épocas	Treino (s)
Dropout 0,5	0,8816 ± 0,0015	0,9542	0,388	0,944	0,063	80	190
Parada antecipada (paciência 10)	0,8798 ± 0,0005	0,9565	0,267	0,901	0,021	12,3	26
L1 1e-4	0,8748 ± 0,0056	0,9510	0,318	0,922	0,047	80	173
Dropout 0,3	0,8716 ± 0,0025	0,9501	0,492	0,965	0,093	80	190
L2 1e-3	0,8699 ± 0,0074	0,9492	0,316	0,920	0,050	80	171
Batch Normalization	0,8699 ± 0,0066	0,9441	0,776	0,971	0,101	80	193
Nenhuma	0,8692 ± 0,0021	0,9443	0,828	0,974	0,105	80	170





- **A combinação venceu em todas as métricas** e reduziu o gap de 0,105 para 0,027. Também custou 55% menos tempo que a rede sem regularização, porque a parada antecipada a encerrou em cerca de 25 épocas.
- **Parada antecipada sozinha** já resolve quase tudo: a perda de validação cai de 0,83 para 0,27 e o treino leva 26 s em vez de 170 s. Ela não impede o overfitting; ela para o treino antes dele.
- **Dropout depende da intensidade.** Com 0,3 o gap ainda é de 0,093; com 0,5 cai para 0,063 e a Macro-F1 sobe para 0,882.

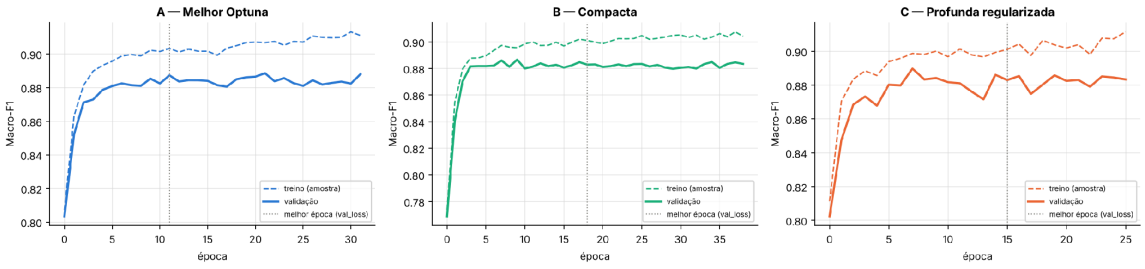
- **L1 e L2** controlam a perda de validação (de 0,83 para 0,32) e reduzem o gap à metade, mas a Macro-F1 fica perto da rede sem regularização.
- **Batch Normalization sozinha não regulariza:** o gap continua em 0,101, e a curva de validação oscila mais. Ela ajuda quando combinada com dropout e parada.

5.3 Três arquiteturas promissoras

Escolhemos três configurações com perfis diferentes de capacidade e custo e treinamos cada uma com 5 sementes:

- **A — Melhor Optuna:** a tentativa nº 30 (2×32, dropout 0,26, BatchNorm, lr 0,0024).
- **B — Compacta:** a tentativa com menos parâmetros entre as que ficaram a até 0,5 p.p. da melhor (nº 28: 1×32, dropout 0,38, BatchNorm, lr 0,0016).
- **C — Profunda regularizada:** 4×512 com a combinação vencedora do experimento 5.2.

Arquitetura	Macro-F1 val. (média ± dp, 5 sementes)	AUC val.	Macro-F1 treino	Gap	Parâmetros	Épocas	Treino (s)	Inferência (µs/amostra)
A — Melhor Optuna (2×32)	0,8832 ± 0,0030	0,9600	0,9018	0,019	11.329	28,8	14,4	13,3
B — Compacta (1×32)	**0,8842 ± 0,0015**	0,9599	0,9050	0,021	**10.177**	33,2	14,7	12,8
C — Profunda regularizada (4×512)	0,8818 ± 0,0037	0,9594	0,9097	0,028	955.393	26,6	80,9	33,6



As três ficam dentro de um desvio-padrão entre sementes. Com 5 sementes, a melhor tentativa do Optuna (0,8873 numa única semente) cai para 0,8832: parte daquele resultado era sorte da semente. **Selecionamos a B** pela maior média na validação. Ela também tem a menor variância, menos

parâmetros e a inferência mais rápida. A C tem 94 vezes mais parâmetros, treina 5,5 vezes mais devagar e não é melhor.

Nas curvas, a validação estabiliza por volta da época 5 nas três redes. Depois disso, o treino continua subindo devagar, e a parada antecipada restaura os pesos da melhor época, marcada pela linha pontilhada.

6. Avaliação final no teste

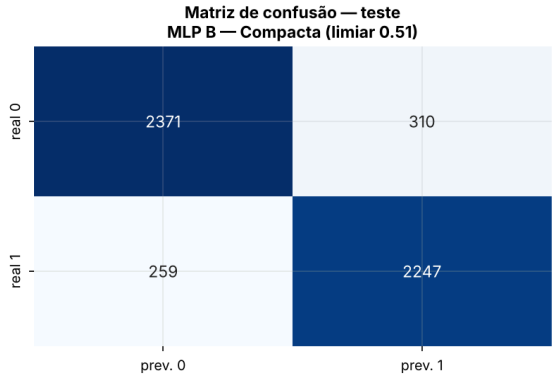
A MLP selecionada (B, 1×32) atingiu **Macro-F1 de 0,8902 e AUC de 0,9648 no teste**, cerca de 1 p.p. acima da regressão logística e do gradient boosting. O limiar de decisão de cada modelo foi escolhido na validação (varredura de 0,20 a 0,80), e o teste foi usado uma única vez, depois de toda a seleção.

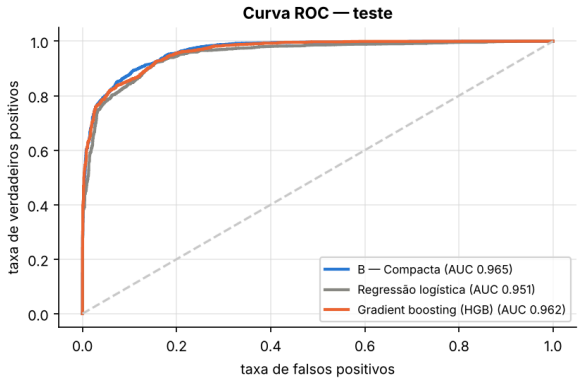
6.1 Métricas no teste

Modelo	Limiar	Macro-F1	Acurácia	Precisão	Revocação	AUC	Log-loss	Parâmetros	Treino (s)	Inferência (µs/amostra)
MLP B — Compacta (1×32), selecionada	0,51	**0,8902**	0,8903	0,8788	0,8966	0,9648	0,239	10.177	14,7	17,4
MLP A — Melhor Optuna (2×32)	0,49	0,8907	0,8907	0,8725	0,9062	0,9654	0,237	11.329	14,4	22,7
MLP C — Profunda regularizada (4×512)	0,48	0,8844	0,8845	0,8809	0,8799	0,9616	0,250	955.393	80,9	34,5
Regressão	0,45	0,8812	0,8812	0,8482	0,9186	0,9514	0,283	—	3,5	0,3

Modelo	Limiar	Macro-F1	Acurácia	Precisão	Revocação	AUC	Log-loss	Parâmetros	Treino (s)	Inferência (µs/amostra)
logística										
Gradient boosting (HGB)	0,44	0,8792	0,8793	0,8348	0,9354	0,9623	0,246	—	3,8	6,1
Classe majoritária	0,50	0,3407	0,5169	0	0	0,5000	7,787	—	0,001	0,01

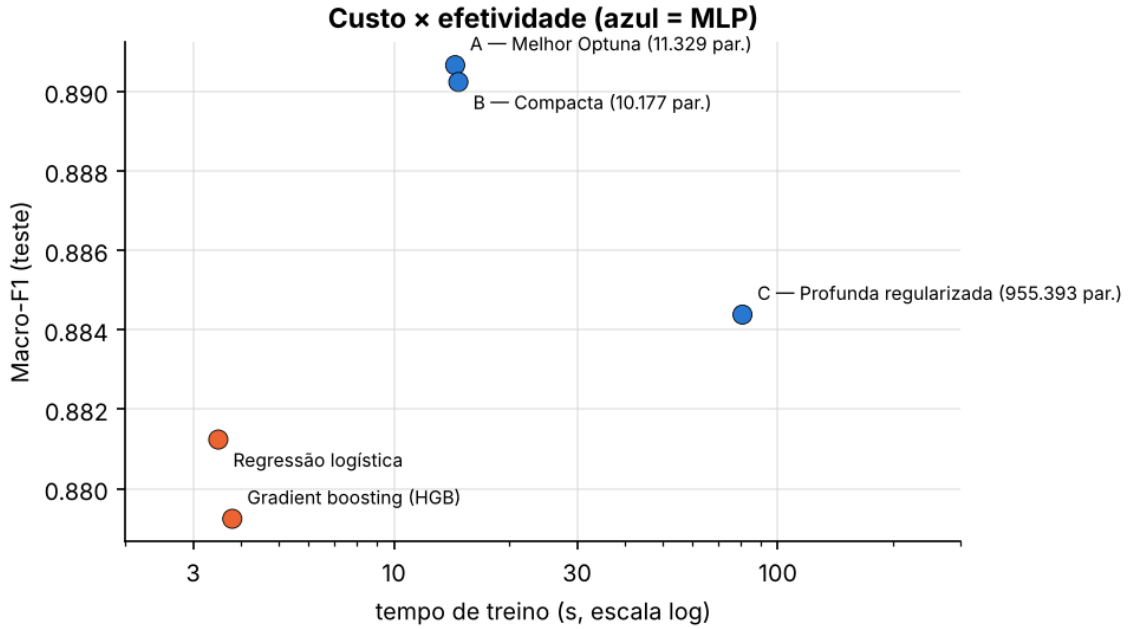
Precisão e revocação referem-se à classe executada. A diferença entre A e B no teste (0,0005) é menor que a variação entre sementes; a escolha da B foi feita na validação e não muda olhando o teste. O ajuste do limiar mexeu pouco: com limiar 0,5 a B teria Macro-F1 de 0,8901.





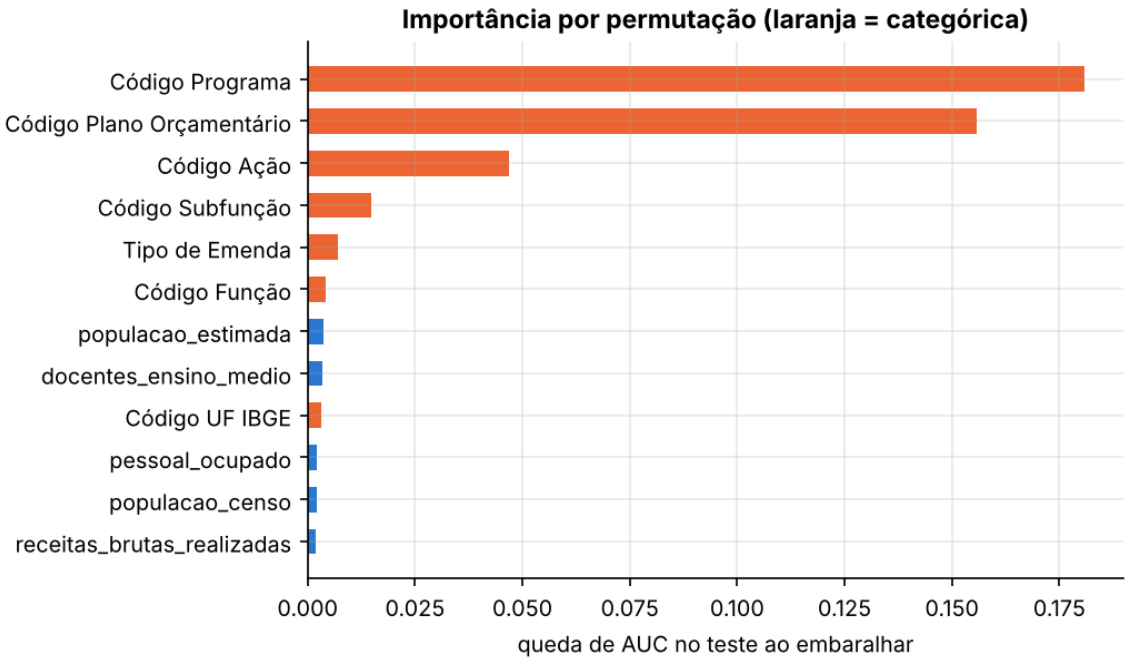
Na matriz de confusão, a MLP B acerta 88,4% das não executadas (2.371 de 2.681) e 89,7% das executadas (2.247 de 2.506). Os erros são equilibrados entre as classes: 310 falsos positivos e 259 falsos negativos. Nas curvas ROC, MLP e gradient boosting quase se sobrepõem; a regressão logística fica um pouco abaixo na faixa de 5% a 20% de falsos positivos.

6.2 Custo × efetividade



- As MLPs pequenas (A e B) dão o melhor desempenho por um custo de cerca de 15 s de treino e 11 mil parâmetros.
- A MLP grande (C) custa 5,5 vezes mais tempo e 94 vezes mais parâmetros para um resultado 0,6 p.p. pior.
- A regressão logística e o gradient boosting treinam 4 vezes mais rápido que a MLP B e ficam cerca de 1 p.p. abaixo em Macro-F1. Na inferência, os três são rápidos o bastante para escorar todas as emendas de um ano em menos de um segundo.
- A busca de hiperparâmetros foi o maior custo: 25 minutos para 40 tentativas em CPU de 2 núcleos, dos quais a poda economizou boa parte ao encerrar 14 tentativas com cerca de 12 épocas. O notebook completo levou cerca de 2 horas.

6.3 Em que a rede se apoia



Embaralhar o código de programa derruba a AUC em 0,181, e o plano orçamentário em 0,156. Seguem a ação (0,047) e a subfunção (0,015). O indicador municipal mais importante, a população estimada, derruba só 0,004. **A rede decide quase só pela classificação orçamentária da emenda**, o que confirma o teste com regressão logística: só os códigos orçamentários dão AUC de 0,946 na validação, e só os atributos da localidade (indicadores municipais, UF, região e bioma), 0,652 (notebook, seção 14.4).

6.4 Sensibilidade à divisão dos dados

Repetimos o treino da MLP B com uma divisão aleatória estratificada por registros, nas mesmas proporções, com 3 sementes:

Divisão	Macro-F1 no teste (média ± dp)	AUC no teste
Agrupada por município (protocolo adotado)	0,8872 ± 0,0047	0,9642
Aleatória por registros	0,8890 ± 0,0015	0,9640

A diferença é de só 0,2 p.p. Como a rede quase não usa os atributos municipais, ela não ganha muito por já ter visto o município no treino. Mesmo assim, a divisão por município é a estimativa honesta para municípios novos e foi mantida.

6.5 Desempenho por subgrupo (teste)

Tipo de emenda	Registros	% executada	Acurácia	Macro-F1
Bancada	205	43,4%	0,946	0,946

Tipo de emenda	Registros	% executada	Acurácia	Macro-F1
Relator	484	46,7%	0,926	0,925
Individual — finalidade definida	4.054	47,9%	0,879	0,879
Comissão	244	19,7%	0,869	0,778
Individual — transferência especial	200	100%	1,000	— (uma só classe)

O pior desempenho é nas emendas de comissão, onde a classe executada é rara (19,7%): a Macro-F1 cai para 0,778. Entre regiões, a Macro-F1 vai de 0,875 no Nordeste a 0,924 no Sul.

7. Limitações e conclusões

7.1 Limitações

- **Os códigos orçamentários carregam o tempo.** O ano da emenda foi removido, mas os códigos de programa mudam a cada Plano Plurianual (PPA). A rede pode estar aprendendo, em parte, *quando* a emenda foi feita, e anos antigos tiveram mais tempo para executar. Isso não é vazamento, pois os códigos existem antes da execução, mas o desempenho em emendas de um PPA futuro, com códigos novos, tende a ser menor. Os códigos novos caem na categoria "infrequente" do codificador.
- **A divisão não é temporal.** Uma validação mais fiel ao uso real treinaria com anos anteriores e testaria nos anos seguintes. O ano foi removido no tratamento inicial; reincluí-lo só para dividir os dados é o próximo passo natural.
- **O alvo é uma fotografia.** Emendas recentes classificadas como não executadas ainda podem vir a ser executadas. A remoção de 2025 e 2026 reduz, mas não elimina, esse ruído no rótulo.
- **Atributos municipais pouco informativos.** Eles mudam pouco o resultado (seção 6.3); dados sobre o convenente, a situação do convênio ou a capacidade de gestão da prefeitura provavelmente ajudariam mais.
- **Classes raras dentro de subgrupos.** Nas emendas de comissão, com só 19,7% de executadas, a Macro-F1 cai para 0,778.
- **Variância e custo.** Os resultados de A, B e C diferem menos que a variação entre sementes. A busca usou 40 tentativas em CPU; um orçamento maior poderia achar configurações um pouco melhores, mas o histórico da busca já mostra um platô entre 0,88 e 0,89.

7.2 Conclusões

1. Uma MLP compacta (1 camada oculta de 32 neurônios, ~10 mil parâmetros) prevê a execução de emendas com **Macro-F1 de 0,890 e AUC de 0,965** em municípios nunca vistos, cerca de 1 p.p. acima da regressão logística e do gradient boosting.
2. **Capacidade não foi o gargalo.** Redes de 5 mil a 950 mil parâmetros chegaram à mesma Macro-F1 máxima; as grandes só sofreram mais overfitting e custaram até 5 vezes mais.

3. **Regularização foi o que mais importou.** O dropout explica 65% da variação entre tentativas do Optuna, e a combinação de dropout, L2, BatchNorm e parada antecipada reduziu o gap treino−validação da rede 4×512 de 0,105 para 0,027.
4. **O sinal está na classificação orçamentária** (programa, plano orçamentário e ação), não no perfil do município. A divisão por município se mostrou pouco sensível por esse motivo.
5. Como trabalho futuro: divisão temporal por ano, embeddings para os códigos de alta cardinalidade no lugar do one-hot e inclusão de dados do convenente e do convênio.